



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN,
RISET, DAN TEKNOLOGI
UNIVERSITAS NEGERI SURABAYA
Kampus Unesa 1, jalan Ketintang Surabaya 60231
Laman: <https://vokasi.unesa.ac.id/> E-mail: vokasi@unesa.ac.id

MODUL 1.

Matematika Komputasi



PROGRAM STUDI MANAJEMEN INFORMATIKA
FAKULTAS VOKASI
UNIVERSITAS NEGERI SURABAYA
2025



Topik

Pengantar Matematika Komputasi.

Tujuan

1. Mampu menguasai konsep dasar matematika komputasi dan menerapkannya dalam pemecahan masalah.
2. Mampu berpikir logis dan sistematis dalam analisis dan pembuktian masalah matematis.
3. Mampu mengembangkan keterampilan pemrograman Python untuk implementasi konsep matematika komputasi.
4. Mampu menggunakan logika dan himpunan dalam menyusun argumen matematis yang valid.
5. Mampu melakukan analisis relasi, fungsi, matriks, dan penerapan induksi matematika dalam masalah komputasi

Petunjuk Praktikum

1. Ikuti langkah-langkah pada bagian praktikum sesuai dengan urutan yang diberikan.
2. Anda dapat menggunakan **Visual Studio Code** atau **IDE lainnya** untuk melakukan praktikum pada modul ini, sesuaikan dengan kondisi komputer Anda.
3. Jawablah semua pertanyaan yang terdapat pada langkah-langkah tertentu di setiap bagian modul praktikum.
4. Dalam setiap langkah pada praktikum terdapat penjelasan yang akan membantu Anda dalam menjawab pertanyaan-pertanyaan pada petunjuk, maka baca dan kerjakanlah semua bagian praktikum dalam modul ini.
5. Dokumentasikan **setiap langkah** sesuai modul yang Anda lakukan di Praktikum dalam bentuk gambar **screenshot** dan berilah **analisis** serta **uraian** penjelasan.
6. Buatlah laporan menggunakan format **cover** yang tersedia.
7. Tulis jawaban dari soal-soal berdasarkan pada petunjuk menjadi sebuah laporan yang dikerjakan menggunakan aplikasi word processing (Word, OpenOffice, atau yang lain yang sejenis), kemudian ekspor sebagai file **PDF** dengan format nama sebagai berikut:
 - [NIM]_NamaLengkapAnda_[Kelas]_MatKom_Praktikum[x].pdf
 - Contoh:
 - 1234567_PaulWalker_2017A_MatKom_Praktikum1.pdf
 - Perhatikan baik-baik format penulisan pada penamaanya.
8. Kumpulkan file PDF tersebut sebagai laporan praktikum kepada dosen pengampu.
9. Selain pada nama file, cantumkan juga **identitas Anda** pada halaman pertama laporan tersebut (cover).
10. Praktikum memiliki alokasi rubrik penilaian yang paling besar, maka pastikan Anda mengerjakan dan mengumpulkan selalu semua tugas praktikum.



PRAKTIKUM [Topik]

[Minggu ke-]

Mata Kuliah:

[ex: Pemrograman Berorientasi Objek]

Oleh:

[Nama]

[NIM]

[Kelas / No. Absen]



PROGRAM STUDI MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

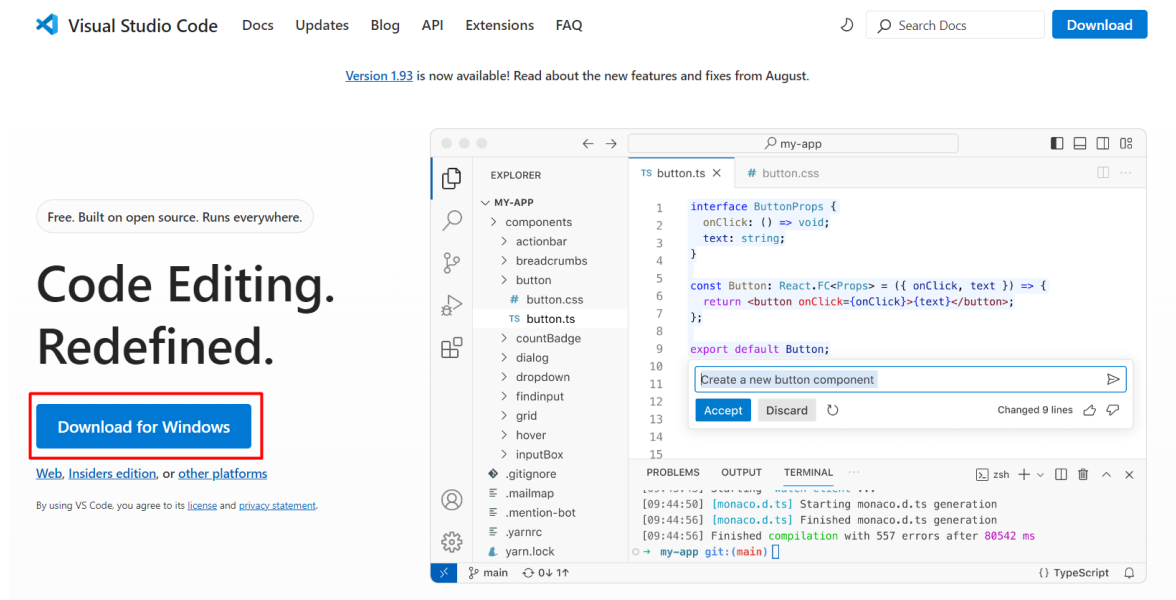
2025

PRAKTIKUM Bagian 1: Proses Instalasi Python Development Environment

Langkah	Praktikum															
1.	https://code.visualstudio.com/docs/python/python-tutorial#_prerequisites To successfully complete this tutorial, you need to first setup your Python development environment. Specifically, this tutorial requires: • Python 3 • VS Code • VS Code Python extension (For additional details on installing extensions, see Extension Marketplace) Silahkan dipelajari secara mandiri, Python Documentation berikut. https://docs.python.org/3.12/tutorial/index.html https://code.visualstudio.com/docs/python/python-tutorial															
2.	How to Install Python Video https://www.youtube.com/watch?v=r1nkXw8ffOk Persiapkan tiga komponen utama pada Langkah 1., dalam mata kuliah ini seperti penjelasan diatas. Download dan install Python 3 terbaru pada perangkat komputer. Download dan Install Python pada perangkat komputer Anda. Klik Link pada Langkah1., untuk mendapatkan akses file yang perlu digunakan.  Active Python Releases For more information visit the Python Developer's Guide. <table><tr><th>Python version</th><th>Maintenance status</th><th>First released</th><th>End of support</th><th>Release schedule</th></tr><tr><td>3.13</td><td>prerelease</td><td>2024-10-01 (planned)</td><td>2029-10</td><td>PEP 719</td></tr><tr><td>3.12</td><td>bugfix</td><td>2023-10-02</td><td>2028-10</td><td>PEP 693</td></tr></table>	Python version	Maintenance status	First released	End of support	Release schedule	3.13	prerelease	2024-10-01 (planned)	2029-10	PEP 719	3.12	bugfix	2023-10-02	2028-10	PEP 693
Python version	Maintenance status	First released	End of support	Release schedule												
3.13	prerelease	2024-10-01 (planned)	2029-10	PEP 719												
3.12	bugfix	2023-10-02	2028-10	PEP 693												

3.

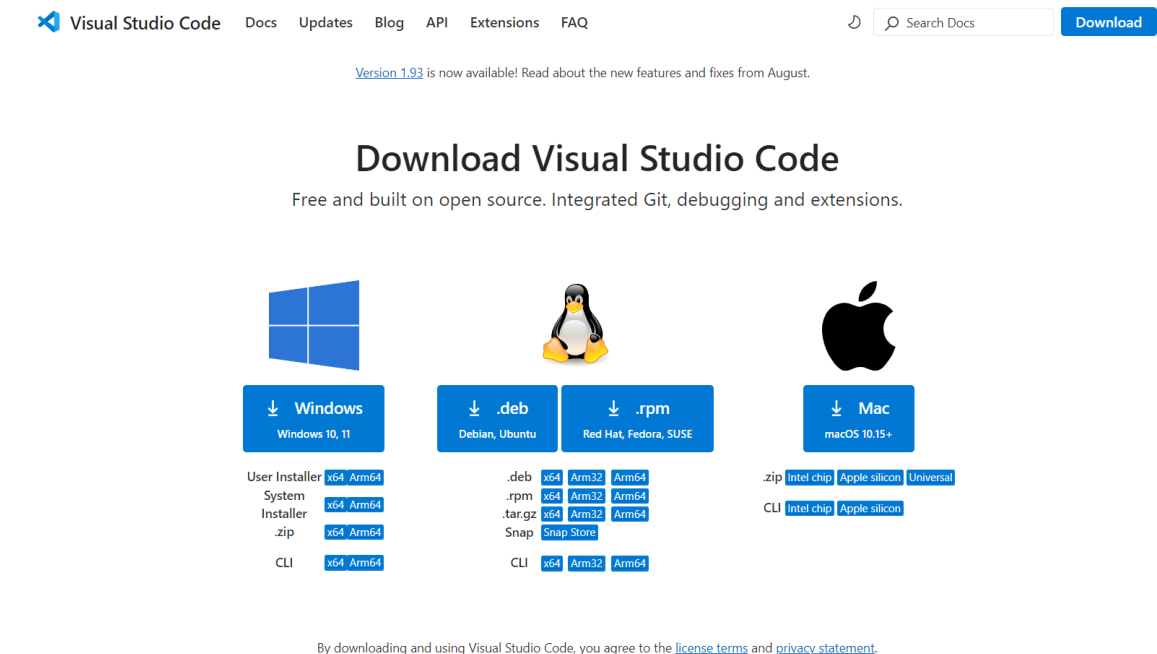
Download Visual Studio Code (VS Code) versi terbaru sebagai Editor Code, dengan akses link dan klik download for windows.



The screenshot shows the Visual Studio Code website. At the top, there are navigation links: Visual Studio Code, Docs, Updates, Blog, API, Extensions, and FAQ. A search bar and a 'Download' button are also visible. Below the navigation, a message states 'Version 1.93 is now available! Read about the new features and fixes from August.' The main heading is 'Code Editing. Redefined.' Below this, a blue button labeled 'Download for Windows' is highlighted with a red rectangular box. Underneath the button, there are links for 'Web, Insiders edition, or other platforms' and a note about agreeing to the license and privacy statement by using VS Code.

4.

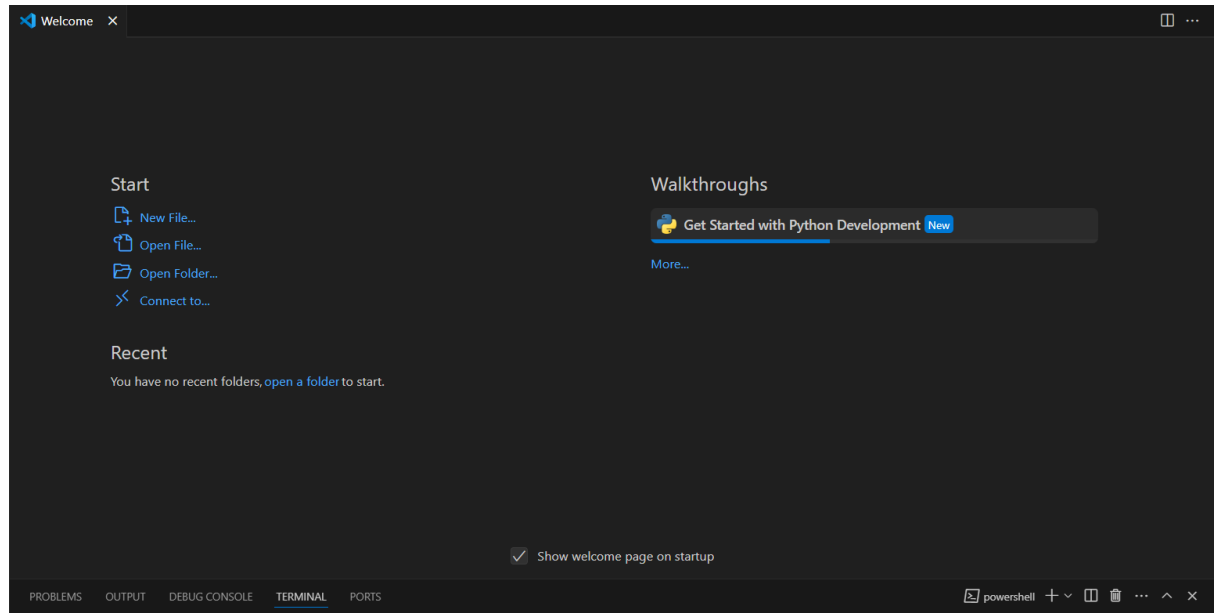
Jika menggunakan platform lain silahkan disesuaikan dengan kebutuhan.



The screenshot shows the 'Download Visual Studio Code' page. It features the Visual Studio Code logo and navigation links. A message states 'Version 1.93 is now available! Read about the new features and fixes from August.' The main heading is 'Download Visual Studio Code' with the subtitle 'Free and built on open source. Integrated Git, debugging and extensions.' Below this, there are three main sections for different operating systems: Windows, Linux (Debian, Ubuntu, Red Hat, Fedora, SUSE), and Mac (macOS 10.15+). Each section has a download button and a list of available download methods and architectures. For Windows, the methods are User Installer, System Installer, .zip, and CLI, all supporting x64 and Arm64 architectures. For Linux, the methods are .deb, .rpm, .tar.gz, Snap, and CLI, supporting x64, Arm32, and Arm64 architectures. For Mac, the methods are .zip and CLI, supporting Intel chip, Apple silicon, and Universal architectures. At the bottom, there is a note about agreeing to the license terms and privacy statement by downloading and using Visual Studio Code.

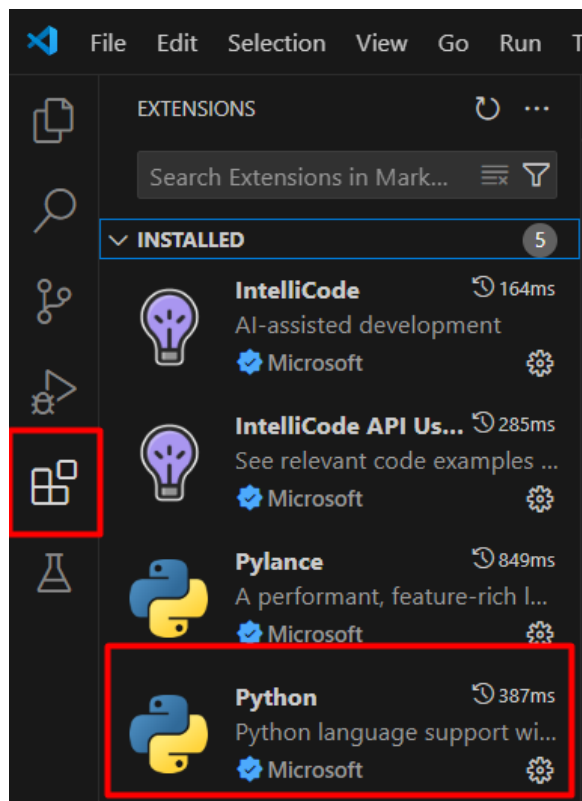
5.

Tampilan awal dari VS Code setelah berhasil diinstall pada perangkat Anda.



6.

Pilih Extensions pada navbar sisi kiri, kemudian search, pilih dan install “Python” for VS Code.



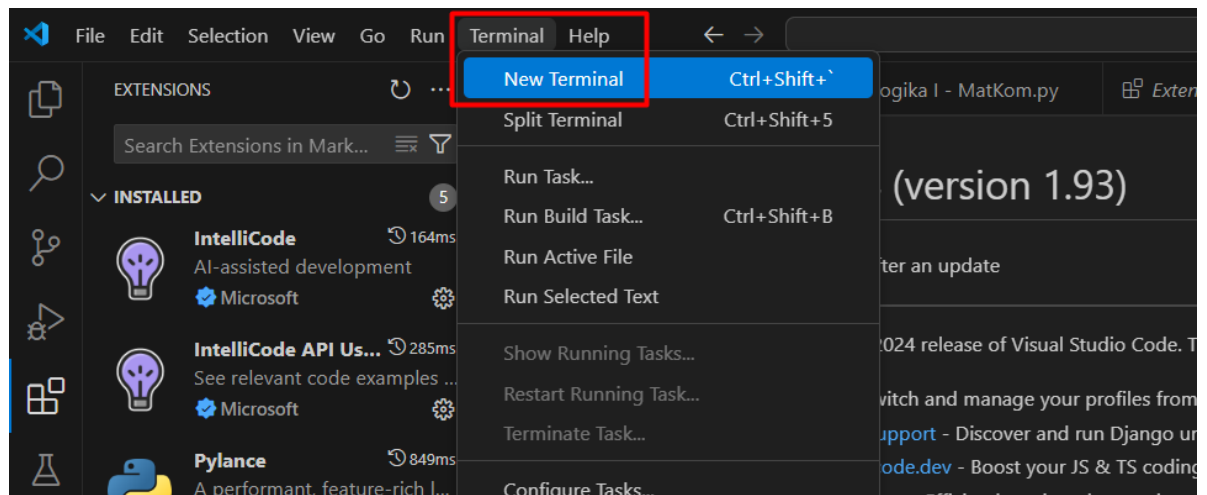
7.

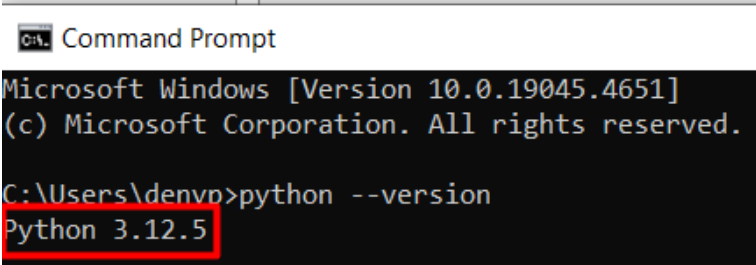
Tampilan jika berhasil menambahkan Extensions Python pada VS Code.



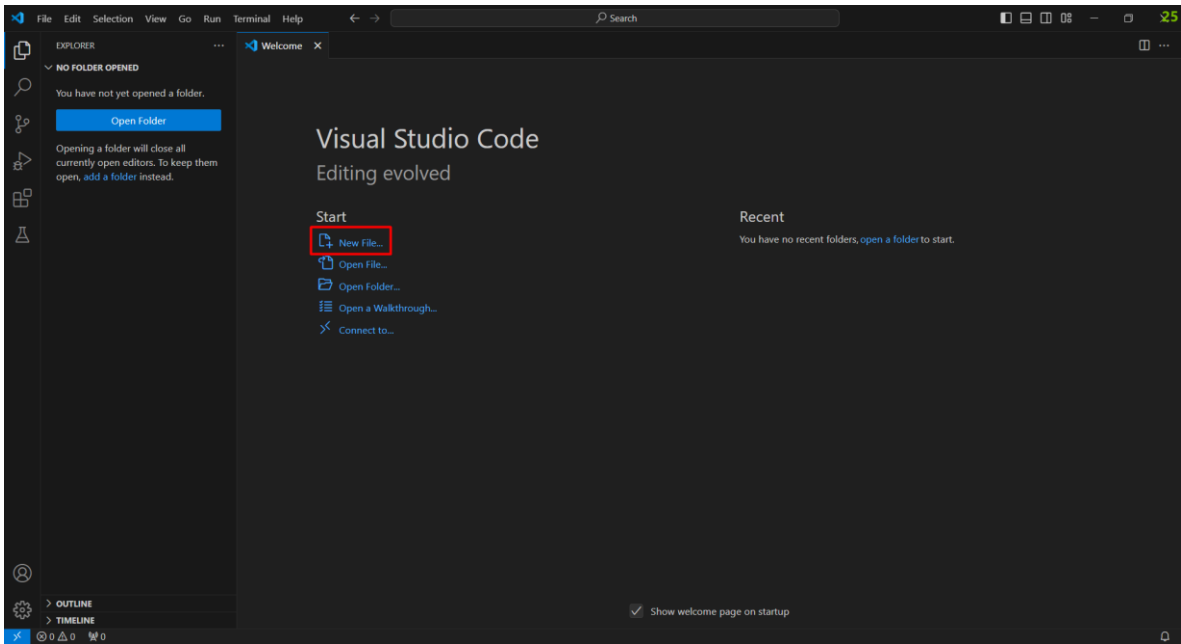
8.

Pilih Terminal pada taskbar → New Terminal.



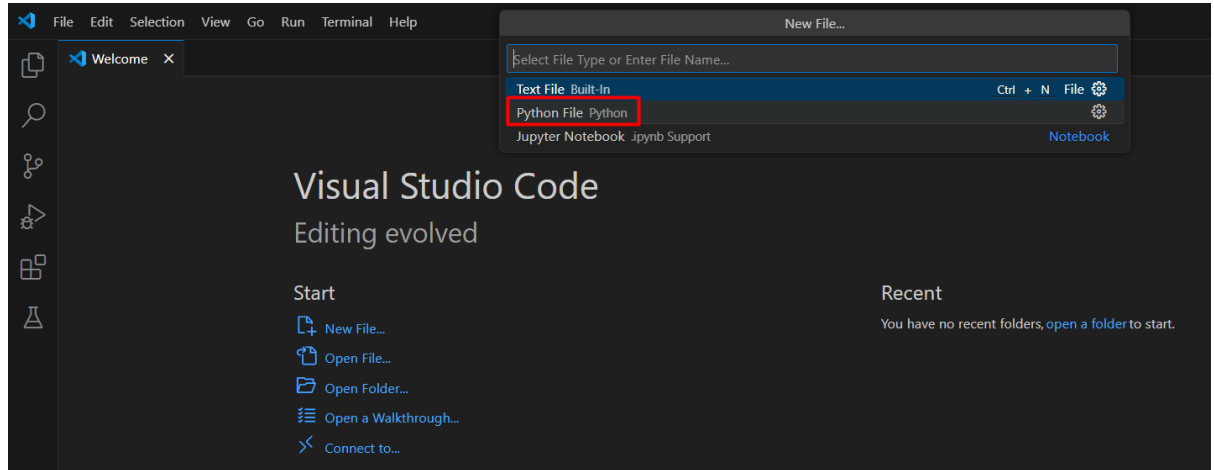
9.	Tuliskan perintah berikut untuk melakukan check pada versi Python yang telah terinstall. 
10.	Lakukan hal yang sama, menggunakan command prompt. 

PRAKTIKUM Bagian 2: Pengantar Teori Matematika Komputasi

Langkah	Praktikum
1.	Buka VS Code yang telah terinstall pada perangkat Anda. Pilih “New File” untuk membuat dokumen baru dengan extensions Python (.py) 

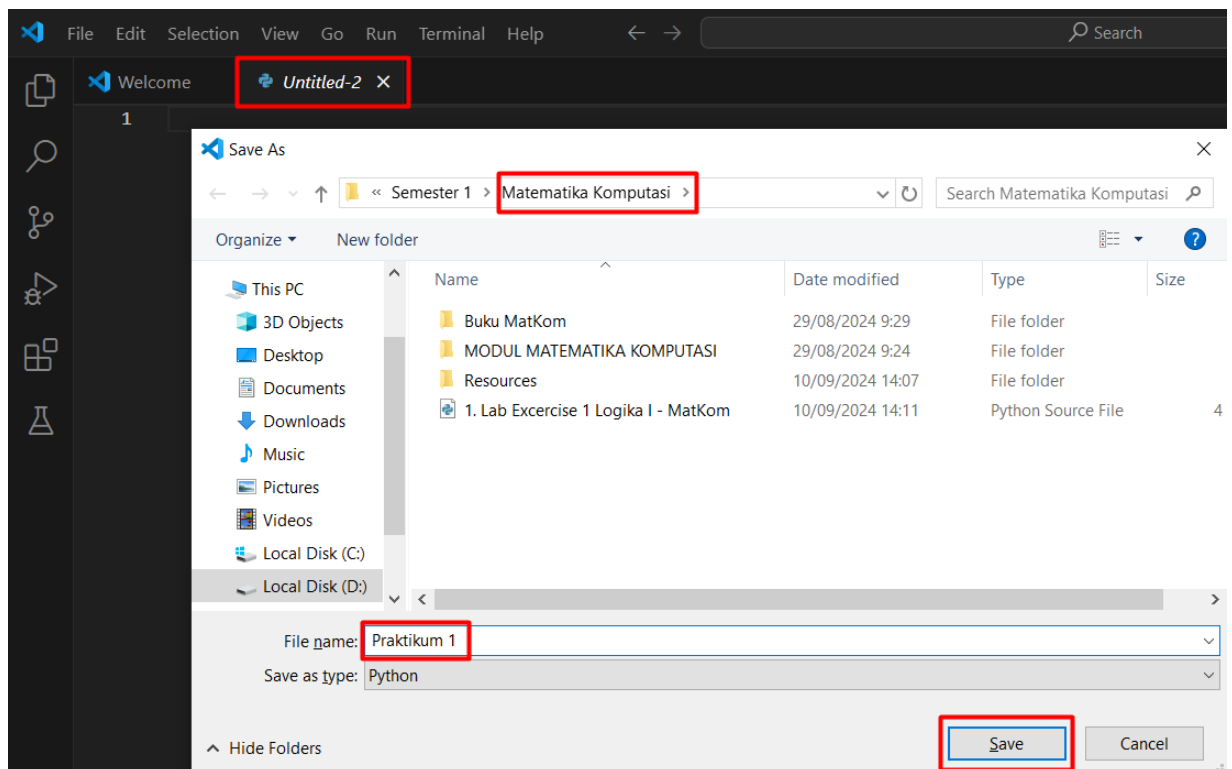
2.

Kemudian pilih Python.

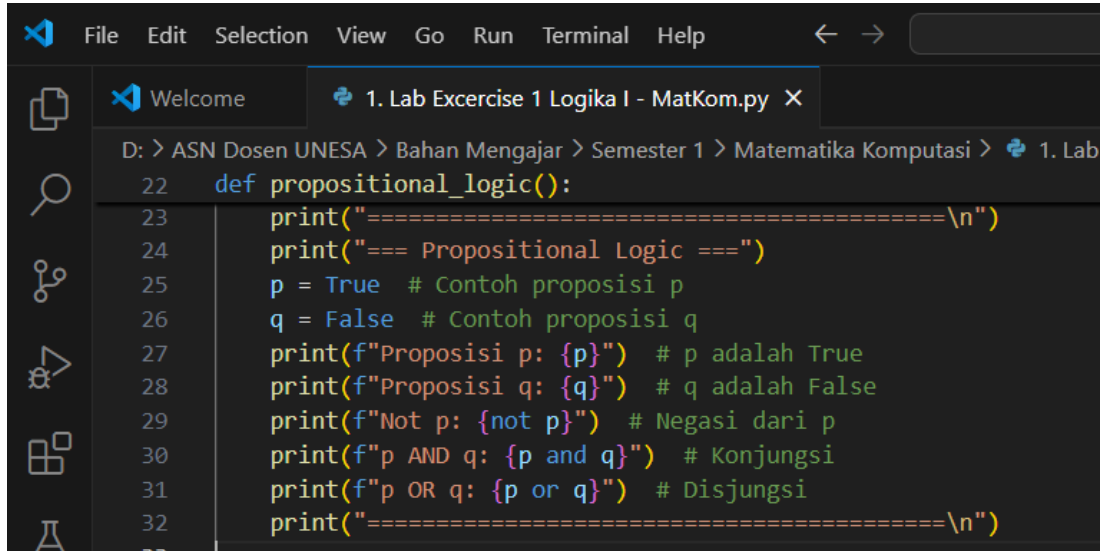


3.

Muncul editor baru dengan ekstensi .py Python, kemudian simpan dengan menekan tombol “Ctrl + S” → Pilih lokasi tempat Anda akan menyimpan File → Ubah nama sesuai dengan keinginan Anda → klik Save.



Ketikkan code program berikut, kemudian running dan analisis Hasilnya.

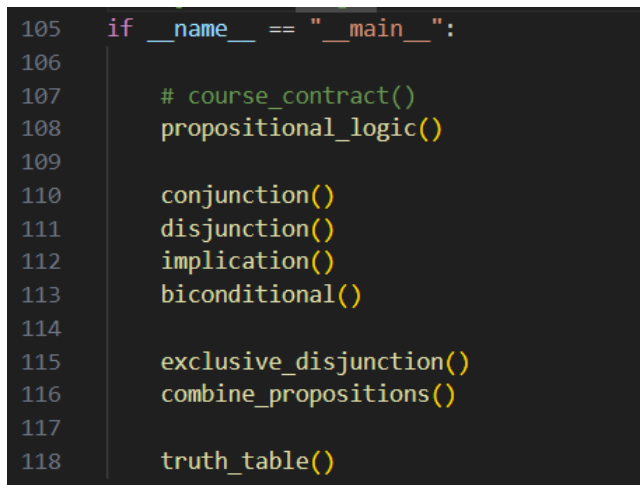


```

22 def propositional_logic():
23     print("=====\n")
24     print("=== Propositional Logic ===")
25     p = True # Contoh proposisi p
26     q = False # Contoh proposisi q
27     print(f"Proposisi p: {p}") # p adalah True
28     print(f"Proposisi q: {q}") # q adalah False
29     print(f"Not p: {not p}") # Negasi dari p
30     print(f"p AND q: {p and q}") # Konjungsi
31     print(f"p OR q: {p or q}") # Disjungsi
32     print("=====\n")
  
```

Sebelum running program, ketikkan perintah untuk memanggil setiap function.

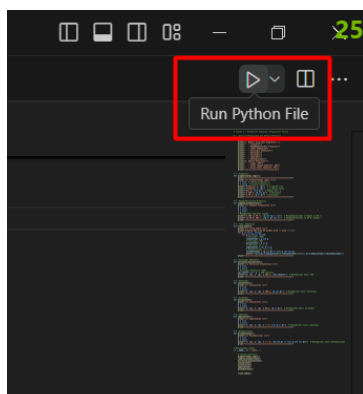
4.



```

105 if __name__ == "__main__":
106
107     # course_contract()
108     propositional_logic()
109
110     conjunction()
111     disjunction()
112     implication()
113     biconditional()
114
115     exclusive_disjunction()
116     combine_propositions()
117
118     truth_table()
  
```

Pada sisi kiri terdapat button untuk running program.



5.	Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 73 def conjunction(): 74 print("=== Conjunction ===") 75 p = True 76 q = False 77 print(f"p: {p}, q: {q}, p AND q: {p and q}") # Menampilkan hasil konjungsi 78 print("=====\n") 79 </pre>
6.	Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 81 def disjunction(): 82 print("=== Disjunction ===") 83 p = True 84 q = False 85 print(f"p: {p}, q: {q}, p OR q: {p or q}") # Menampilkan hasil disjungsi 86 print("=====\n") 87 </pre>
7.	Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 89 def implication(): 90 print("=== Implication ===") 91 p = True 92 q = False 93 print(f"p: {p}, q: {q}, p -> q: {not p or q}") # Menampilkan hasil implikasi 94 print("=====\n") 95 </pre>
8.	Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 97 def biconditional(): 98 print("=== Biconditional ===") 99 p = True 100 q = False 101 print(f"p: {p}, q: {q}, p <-> q: {(p and q) or (not p and not q)}") 102 print("=====\n") 103 </pre>

9.	Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 63 def exclusive_disjunction(): 64 print("=== Exclusive Disjunction ===") 65 p = True 66 q = False 67 # Disjungsi Eksklusif (XOR) 68 xor_result = p ^ q # XOR operator 69 print(f"p: {p}, q: {q}, p XOR q: {xor_result}") # Menampilkan hasil XOR 70 print("=====\n") 71 </pre>
10.	Pada code program ini terdapat tambahan satu variabel yaitu “r”, digunakan untuk apa? Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> 35 def combine_propositions(): 36 print("=== Combine Proposition ===") 37 p = True 38 q = False 39 r = True 40 # Menggunakan operator logika 41 print(f"p AND (q OR r): {p and (q or r)}") # Mengkombi 42 print(f"(p OR q) AND r: {(p or q) and r}") # Mengkombi 43 print("=====\n") 44 </pre>
11.	Code Program ini merupakan function dari Truth Table pada beberapa teori matematika komputasi. Ketikkan code program berikut, kemudian running dan analisis hasilnya. <pre> def truth_table(): print("=== Truth Table ===") print("p\tq\tp AND q\t\tp OR q\t\tNot p\t\tp -> q\t\tp <-> q") for p in [True, False]: for q in [True, False]: conjunction = p and q disjunction = p or q implication = not p or q biconditional = (p and q) or (not p and not q) print(f"{p}\t{q}\t{conjunction}\t\t{disjunction}\t\t{not p}\t\t{implication}\t\t{biconditional}") print("=====\n") </pre>



12.

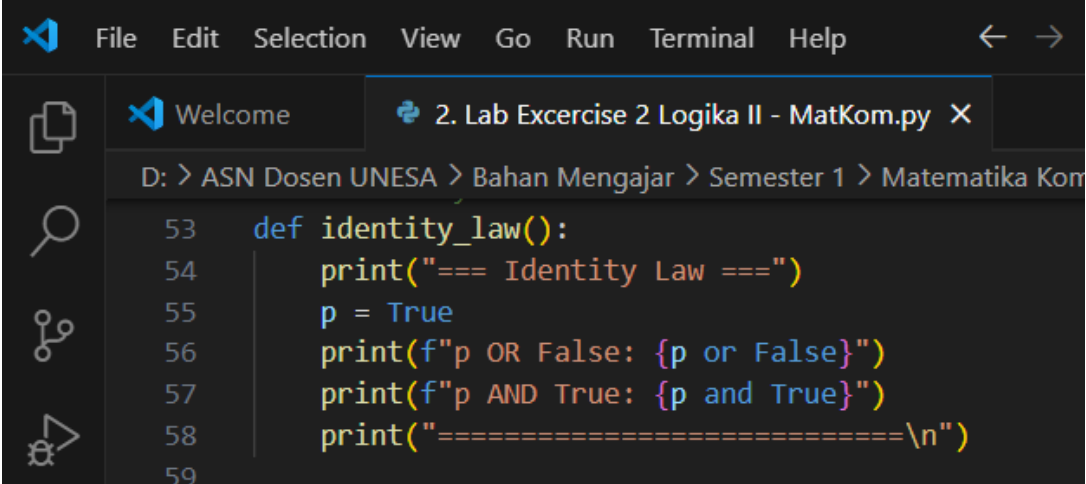
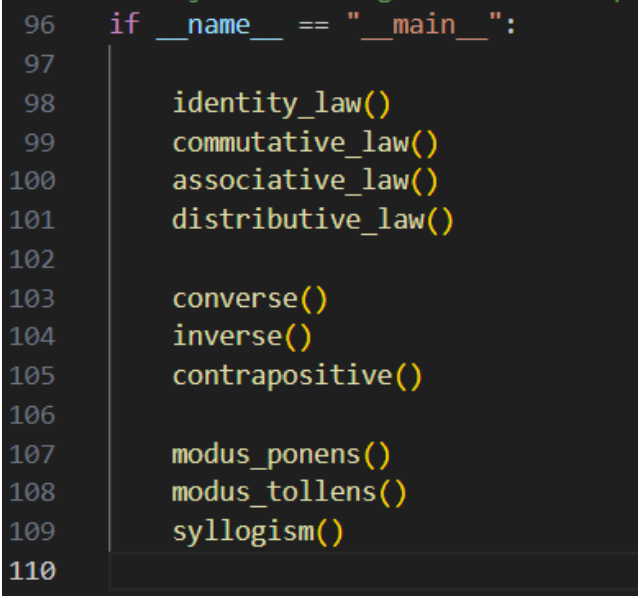
Code berikut merupakan Function yang digunakan untuk memanggil block code pada setiap teori dari Matematika Komputasi. Running program tersebut, dan analisislah hasilnya.

```
105 if __name__ == "__main__":  
106  
107     # course_contract()  
108     propositional_logic()  
109  
110     conjunction()  
111     disjunction()  
112     implication()  
113     biconditional()  
114  
115     exclusive_disjunction()  
116     combine_propositions()  
117  
118     truth_table()  
119
```

PRAKTIKUM Bagian 3: Tugas Praktikum Pengantar Teori Matematika Komputasi

Langkah	Praktikum
1.	Pilihlah salah dua dari penerapan teori Matematika Komputasi yang baru saja dilakukan.
2.	Ubahlah nilainya yang semula True → menjadi False, atau sebaliknya. Ex; (p = True, q=False → p = False, q= True, p = True, q=False → p = False, q= False, p = True, q=False → p = True, q= True)
3.	Screenshoot code dan hasil outputnya, serta analisis hasilnya.

PRAKTIKUM Bagian 4: Penerapan Hukum Propositional Logic

Langkah	Praktikum
	Hukum proposisi mempunyai empat jenis, sebagai berikut. Tuliskan code program Teori Identitas berikut, dan analisis hasilnya.  Sebelum running program, Tuliskan perintah untuk memanggil setiap function. 1. 

2.	Tuliskan code program Teori Komutatif berikut, dan analisis hasilnya. <pre> 61 def commutative_law(): 62 print("=== Commutative Law ===") 63 p = True 64 q = False 65 print(f"p OR q: {p or q}") 66 print(f"q OR p: {q or p}") 67 print(f"p AND q: {p and q}") 68 print(f"q AND p: {q and p}") 69 print("=====\\n") 70 </pre>
3.	Tuliskan code program Teori Asosiatif berikut, dan analisis hasilnya. <pre> 72 def associative_law(): 73 print("=== Associative Law ===") 74 p = True 75 q = False 76 r = True 77 print(f"(p OR q) OR r: {(p or q) or r}") 78 print(f"p OR (q OR r): {p or (q or r)}") 79 print(f"(p AND q) AND r: {(p and q) and r}") 80 print(f"p AND (q AND r): {p and (q and r)}") 81 print("=====\\n") 82 </pre>
4.	Tuliskan code program Teori Distributif berikut, dan analisis hasilnya. <pre> 84 def distributive_law(): 85 print("=== Distributive Law ===") 86 p = True 87 q = False 88 r = True 89 print(f"p AND (q OR r): {p and (q or r)}") 90 print(f"(p AND q) OR (p AND r): {(p and q) or (p and r)}") 91 print(f"p OR (q AND r): {p or (q and r)}") 92 print(f"(p OR q) AND (p OR r): {(p or q) and (p or r)}") 93 print("=====\\n") </pre>



PRAKTIKUM Bagian 5: Penerapan Teori Variants of Conditional Propositions

Langkah	Praktikum
1.	Tuliskan code program Varian Proposisi Bersyarat pada Teori Converse berikut, dan analisis hasilnya. <pre>4 def converse(): 5 print("=== Converse (Konversi) ===") 6 p = True 7 q = False 8 print(f"q -> p: {q} -> {p}") 9 print("=====\n") 10</pre>
2.	Tuliskan code program Varian Proposisi Bersyarat pada Teori Inverse berikut, dan analisis hasilnya. <pre>12 def inverse(): 13 print("=== Inverse (Invers) ===") 14 p = True 15 q = False 16 print(f"~p -> ~q: {not p} -> {not q}") 17 print("=====\n") 18</pre>
3.	Tuliskan code program Varian Proposisi Bersyarat pada Teori Contrapositive berikut, dan analisis hasilnya. <pre>20 def contrapositive(): 21 print("=== Contrapositive (Kontraposisi) ===") 22 p = True 23 q = False 24 print(f"~q -> ~p: {not q} -> {not p}") 25 print("=====\n") 26</pre>

PRAKTIKUM Bagian 6: Penerapan Teori Logical Inference and Validity

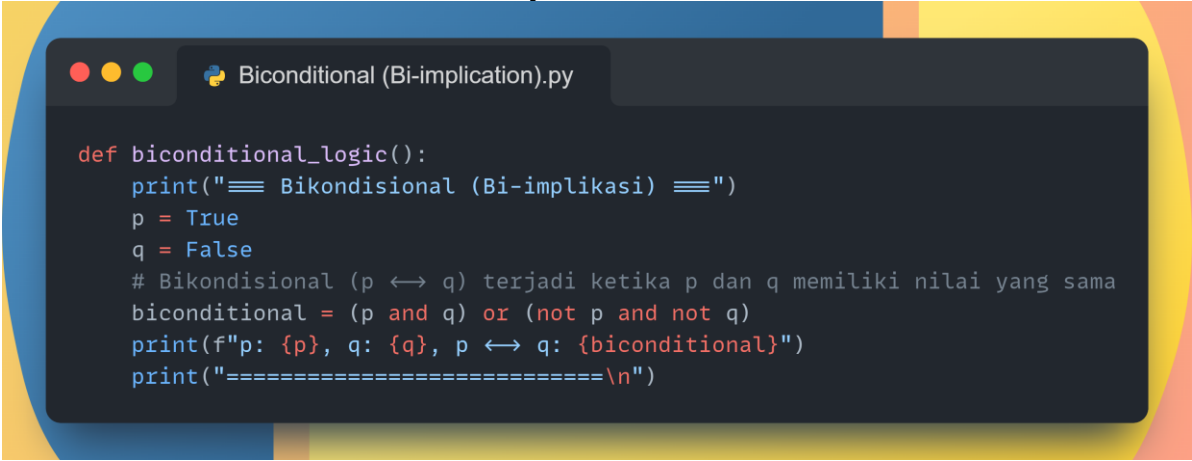
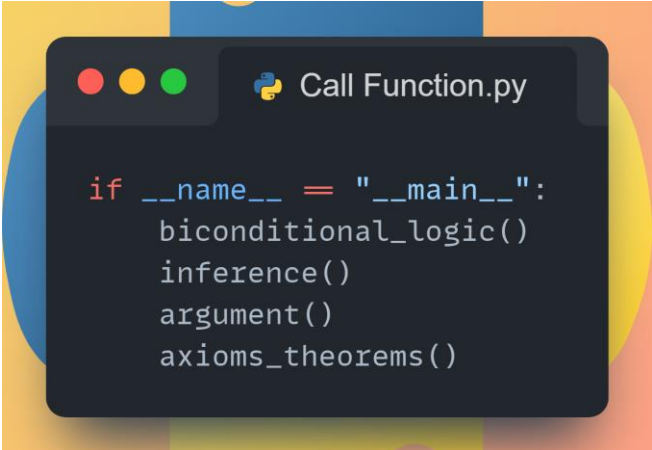
Langkah	Praktikum
1.	Tuliskan code program Teori Logical Inference and Validity pada Teori Modus Ponens berikut, dan analisis hasilnya. <pre> 28 def modus_ponens(): 29 print("=== Modus Ponens ===") 30 p = True 31 q = True 32 print(f"q: {q}") 33 print("=====\n") 34 </pre>
2.	Tuliskan code program Teori Logical Inference and Validity pada Teori Modus Tollens berikut, dan analisis hasilnya. <pre> 36 def modus_tollens(): 37 print("=== Modus Tollens ===") 38 p = True 39 q = False 40 print(f"~p: {not p}") 41 print("=====\n") 42 </pre>
3.	Tuliskan code program Teori Logical Inference and Validity pada Teori Silogisme berikut, dan analisis hasilnya. <pre> 44 def syllogism(): 45 print("=== Syllogism ===") 46 p = True 47 q = True 48 r = False 49 print(f"p -> r: {p} -> {r}") 50 print("=====\n") 51 </pre>

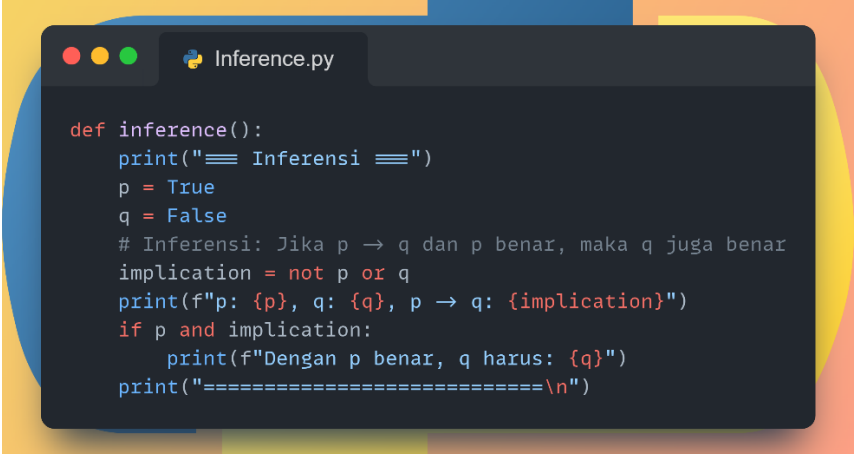


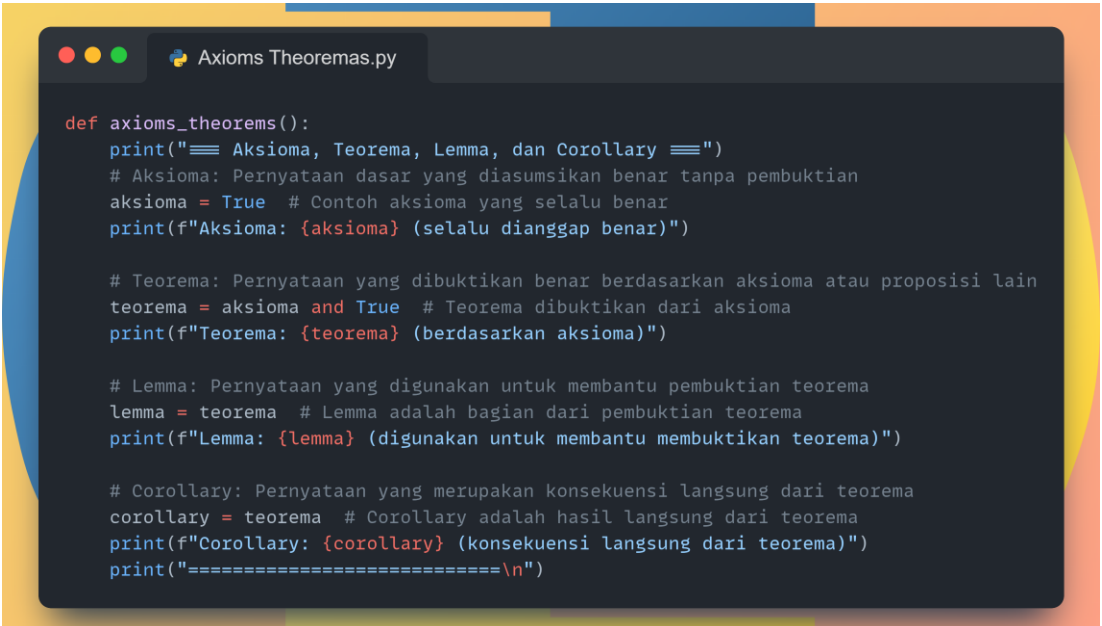
PRAKTIKUM Bagian 7: Tugas Praktikum Propositional Logic and Variants of Implication.

Langkah	Praktikum
1.	Buatlah program Python yang menampilkan hasil dari penerapan Teori Converse, Inverse, dan Contrapositive dari sebuah proposisi. Proposisi dapat diambil dari input nilai dan tampilkan hasilnya di konsol terminal.
2.	Switch input nilai boolean (true/false) untuk proposisi dari p dan q. Implementasikan fungsi untuk menghitung: • Converse ($q \rightarrow p$) • Inverse ($\sim p \rightarrow \sim q$) • Contrapositive ($\sim q \rightarrow \sim p$) Ex; ($p = \text{True}, q = \text{False} \rightarrow p = \text{False}, q = \text{True}$)
3.	Tampilkan hasilnya di Terminal, screenshot dan analisis.

PRAKTIKUM Bagian 8: Penerapan Teori Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas, dan Corollaries

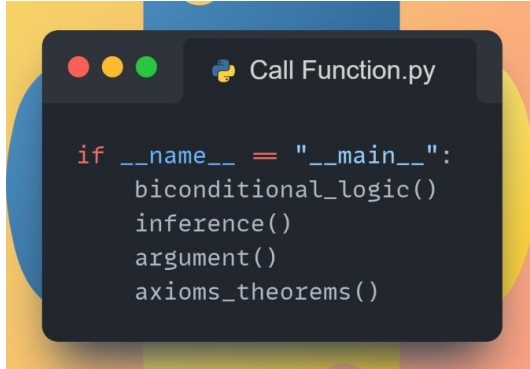
Langkah	Praktikum
1.	Bikondisional (Bi-implikasi): Bikondisional, atau bi-implikasi, merupakan proposisi logika yang bernilai benar jika kedua pernyataan yang dibandingkan memiliki nilai yang sama. Dengan kata lain, $p \leftrightarrow q$ bernilai benar ketika p dan q keduanya benar, atau p dan q keduanya salah. $p \leftrightarrow q$ setara dengan $(p \rightarrow q) \text{ AND } (q \rightarrow p)$ Implementasi logika bikondisional di mana $p \leftrightarrow q$ terjadi jika dan hanya jika p dan q memiliki nilai yang sama. Ketikkan code berikut dan analisis hasilnya. 
2.	Ketikkan code berikut untuk memanggil function yang diperlukan, analisis mengapa terdapat error? 

3.	Inferensi: Inferensi adalah proses menarik kesimpulan logis berdasarkan premis yang telah diberikan. Jika premis benar, maka kesimpulan juga akan benar. Jika $p \rightarrow q$ dan p benar, maka q juga benar (modus ponens). Implementasi inferensi sederhana, di mana kita memverifikasi bahwa jika $p \rightarrow q$ benar dan p benar, maka q juga benar. Ketikkan code berikut dan analisis hasilnya.  <pre>def inference(): print("=== Inferensi ===") p = True q = False # Inferensi: Jika p → q dan p benar, maka q juga benar implication = not p or q print(f"p: {p}, q: {q}, p → q: {implication}") if p and implication: print(f"Dengan p benar, q harus: {q}") print("=====\n")</pre>
4.	Argumen: Argumen logika terdiri dari satu atau lebih premis yang digunakan untuk mendukung suatu kesimpulan. • Premis 1: $p \rightarrow q$ • Premis 2: p • Kesimpulan: q (berdasarkan Modus Ponens) Implementasi argumen logika dengan dua premis dan kesimpulan ($p \rightarrow q$ dan $q \rightarrow r$, maka $p \rightarrow r$). Ketikkan code berikut dan analisis hasilnya.

	 <pre> def argument(): print("=== Argumen ===") p = True q = False r = True # Argumen dalam logika, contoh sederhana: # Jika p → q dan q → r, maka p → r premise_1 = not p or q # p → q premise_2 = not q or r # q → r conclusion = not p or r # p → r print(f"Premis 1 (p → q): {premise_1}") print(f"Premis 2 (q → r): {premise_2}") print(f"Kesimpulan (p → r): {conclusion}") print("=====\n") </pre>
5.	Aksioma, Teorema, Lemma, dan Corollary • Aksioma: Pernyataan yang diterima sebagai kebenaran tanpa perlu pembuktian. • Teorema: Pernyataan yang bisa dibuktikan kebenarannya berdasarkan aksioma. • Lemma: Teorema kecil yang digunakan sebagai langkah dalam pembuktian teorema yang lebih besar. • Corollary: Kesimpulan langsung dari teorema yang sudah terbukti. Demonstrasi penerapan dari pernyataan dasar tentang aksioma, teorema, lemma, dan corollary, di mana aksioma dianggap benar tanpa pembuktian, dan teorema, lemma, serta corollary diturunkan dari aksioma. Ketikkan code berikut dan analisis hasilnya.  <pre> def axioms_theorems(): print("=== Aksioma, Teorema, Lemma, dan Corollary ===") # Aksioma: Pernyataan dasar yang diasumsikan benar tanpa pembuktian aksioma = True # Contoh aksioma yang selalu benar print(f"Aksioma: {aksioma} (selalu dianggap benar)") # Teorema: Pernyataan yang dibuktikan benar berdasarkan aksioma atau proposisi lain teorema = aksioma and True # Teorema dibuktikan dari aksioma print(f"Teorema: {teorema} (berdasarkan aksioma)") # Lemma: Pernyataan yang digunakan untuk membantu pembuktian teorema lemma = teorema # Lemma adalah bagian dari pembuktian teorema print(f"Lemma: {lemma} (digunakan untuk membantu membuktikan teorema)") # Corollary: Pernyataan yang merupakan konsekuensi langsung dari teorema corollary = teorema # Corollary adalah hasil langsung dari teorema print(f"Corollary: {corollary} (konsekuensi langsung dari teorema)") print("=====\n") </pre>

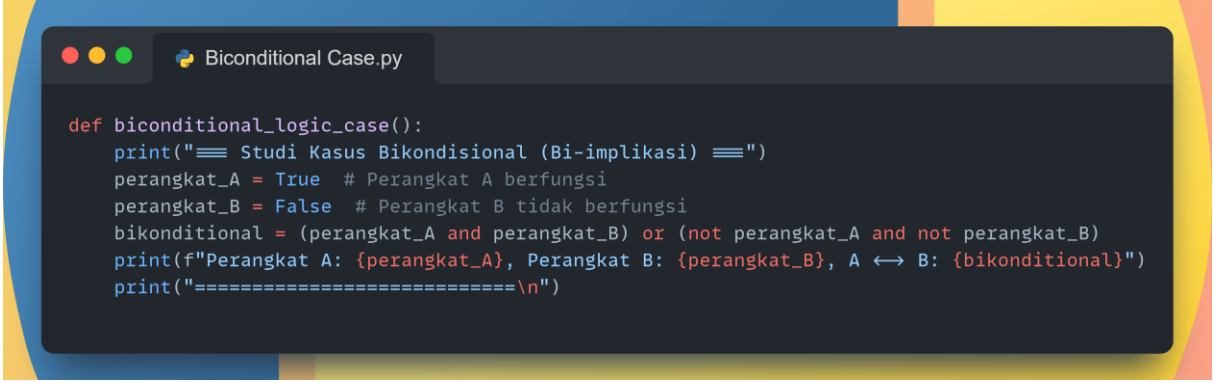
6.

Ketikkan code berikut untuk memanggil function yang diperlukan, dan analisis hasilnya.



```
if __name__ == "__main__":  
    biconditional_logic()  
    inference()  
    argument()  
    axioms_theorems()
```

PRAKTIKUM Bagian 9: Studi Kasus dalam Teori Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas and Corollaries

Langkah	Praktikum
4.	Studi Kasus Bikondisional (Bi-implikasi) Misalkan Anda memiliki dua perangkat jaringan, p dan q. - p: Perangkat A berfungsi dengan baik. - q: Perangkat B berfungsi dengan baik. Dalam logika bikondisional, pernyataan ini berarti: "Perangkat A berfungsi jika dan hanya jika perangkat B berfungsi." Dimana artinya, kedua perangkat harus berfungsi secara bersamaan atau keduanya tidak berfungsi. Jika salah satu perangkat berfungsi sementara yang lain tidak, maka pernyataan ini akan salah. Ketikkan code berikut dan tampilkan hasilnya.  <pre>def biconditional_logic_case(): print("== Studi Kasus Bikondisional (Bi-implikasi) ==") perangkat_A = True # Perangkat A berfungsi perangkat_B = False # Perangkat B tidak berfungsi bikondisional = (perangkat_A and perangkat_B) or (not perangkat_A and not perangkat_B) print(f"Perangkat A: {perangkat_A}, Perangkat B: {perangkat_B}, A ↔ B: {bikondisional}") print("===== \n")</pre>
5.	Studi Kasus Inferensi Misalkan dalam suatu sistem keamanan terdapat pernyataan, jika sensor mendeteksi gerakan (p), maka alarm berbunyi (q). - p: Sensor mendeteksi gerakan. - q: Alarm berbunyi. Jika sensor mendeteksi gerakan dan kita tahu bahwa sensor ini selalu bekerja dengan baik (validitas dari $p \rightarrow q$), maka dapat disimpulkan bahwa alarm harus berbunyi. Ketikkan code berikut dan tampilkan hasilnya.

```

def inference_case():
    print("≡≡≡ Studi Kasus Inferensi ≡≡≡")
    sensor_gerak = True # Sensor mendeteksi gerakan
    alarm_berbunyi = False # Alarm belum berbunyi
    implication = not sensor_gerak or alarm_berbunyi
    print(f"Sensor mendeteksi gerakan: {sensor_gerak}, Alarm berbunyi: {alarm_berbunyi}")
    print(f"p → q: {implication}")
    if sensor_gerak and implication:
        print("Alarm harus berbunyi!")
    else:
        print("Sistem gagal mendeteksi pergerakan!")
    print("=====\n")

```

Studi Kasus Argumen

Misalkan dalam sebuah proses bisnis,

- Jika tim menyelesaikan proyek tepat waktu (p), maka klien akan puas (q).
- Jika klien puas (q), maka perusahaan akan menerima bonus (r).

Dengan dua premis di atas, dapat disimpulkan bahwa jika proyek selesai tepat waktu, perusahaan akan menerima bonus.

Ketikkan code berikut dan tampilkan hasilnya.

6.

```

def argument_case():
    print("≡≡≡ Studi Kasus Argumen ≡≡≡")
    proyek_selesai_tepat_waktu = True # Tim menyelesaikan proyek tepat waktu
    klien_puas = False # Klien tidak puas
    perusahaan_terima_bonus = True # Perusahaan menerima bonus
    premise_1 = not proyek_selesai_tepat_waktu or klien_puas
    premise_2 = not klien_puas or perusahaan_terima_bonus
    conclusion = not proyek_selesai_tepat_waktu or perusahaan_terima_bonus
    print(f"Premis 1 (p → q): {premise_1}")
    print(f"Premis 2 (q → r): {premise_2}")
    print(f"Kesimpulan (p → r): {conclusion}")
    print("=====\n")

```

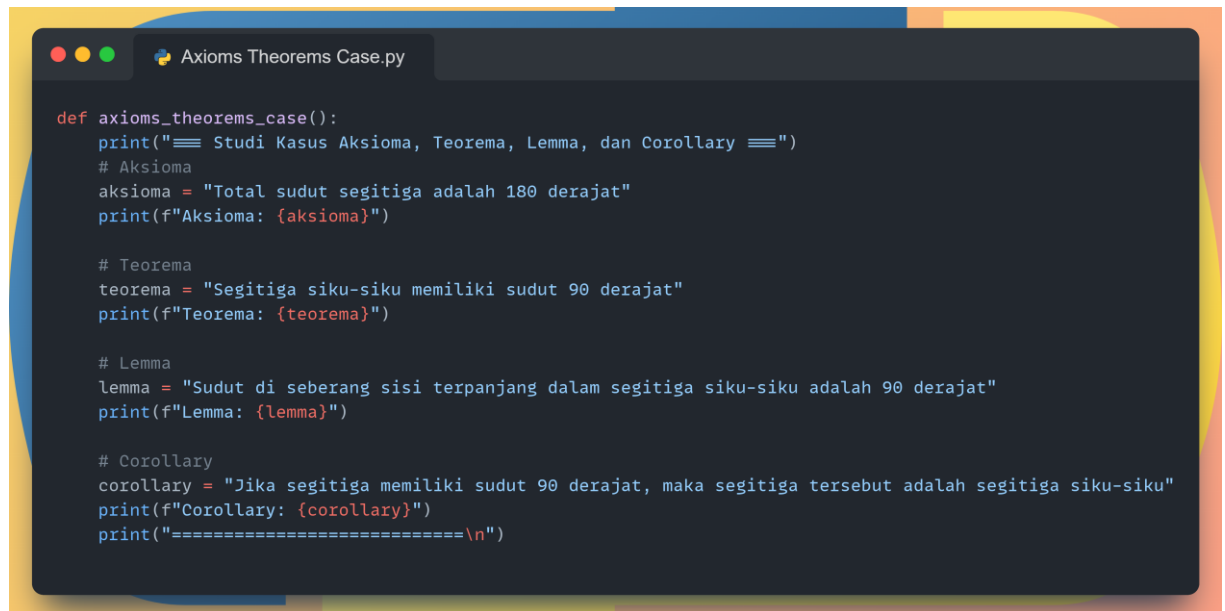

Studi Kasus Aksioma, Teorema, Lemma, dan Corollary

Dalam suatu sistem matematika terdapat pernyataan,

- **Aksioma:** Setiap segitiga memiliki total sudut 180 derajat (diasumsikan benar tanpa bukti).
- **Teorema:** Jika segitiga itu siku-siku, maka salah satu sudutnya adalah 90 derajat (dibuktikan dari aksioma).
- **Lemma:** Sudut di seberang sisi terpanjang dalam segitiga siku-siku adalah 90 derajat (dibuktikan untuk membuktikan teorema).
- **Corollary:** Jika sudut segitiga adalah 90 derajat, maka segitiga itu adalah segitiga siku-siku (konsekuensi langsung dari teorema).

Ketikkan code berikut dan tampilkan hasilnya.

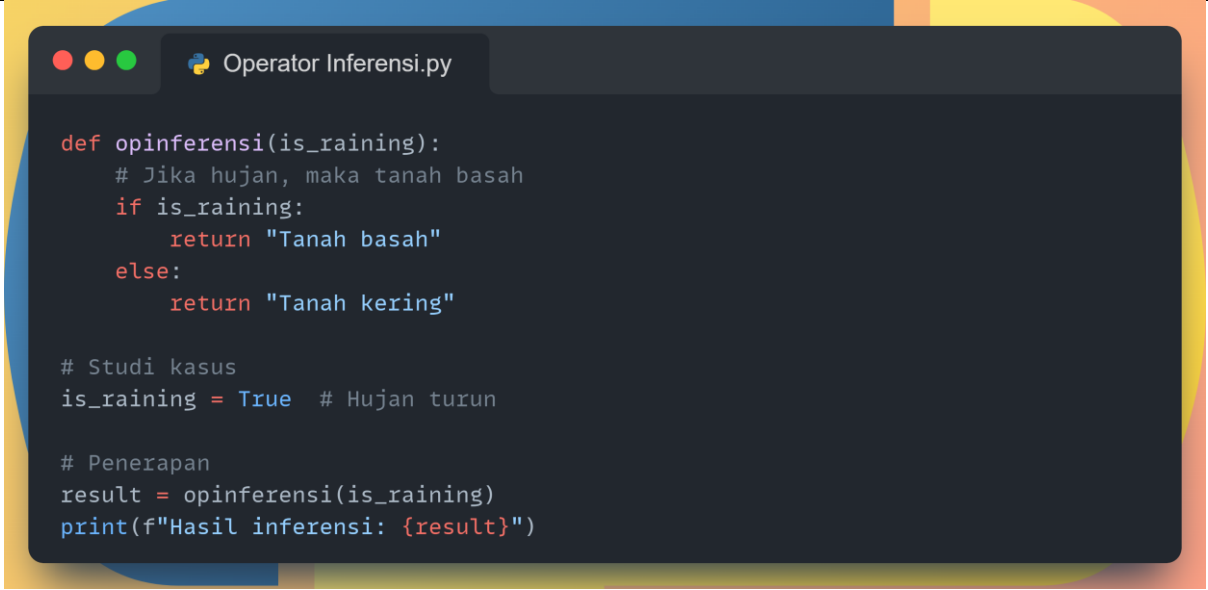
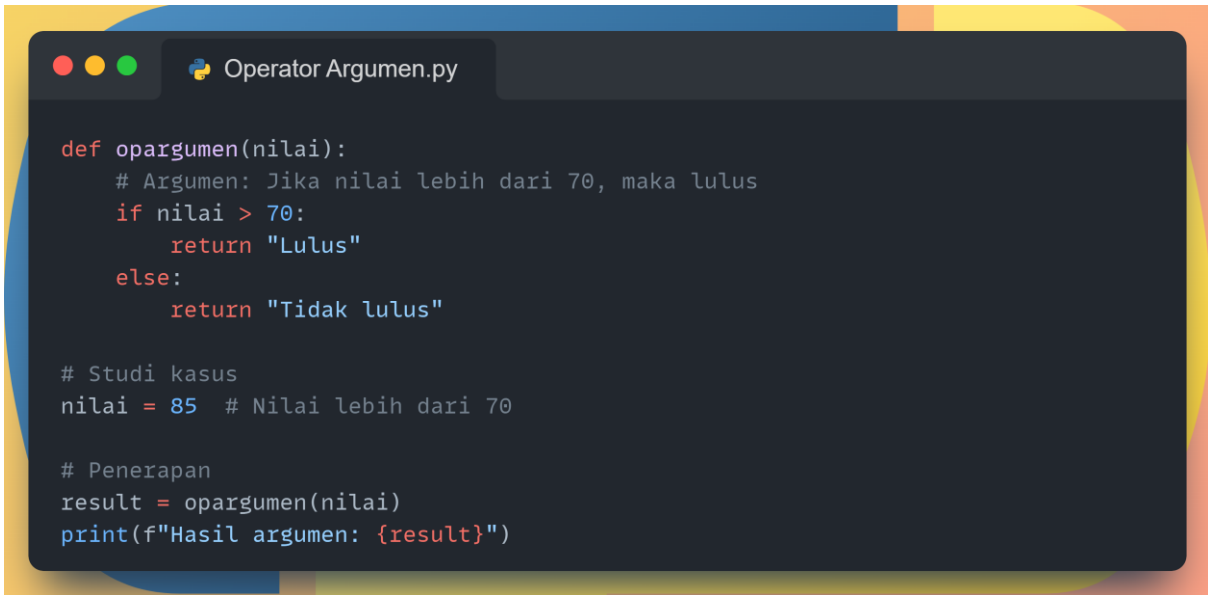
7.



```
def axioms_theorems_case():  
    print("=== Studi Kasus Aksioma, Teorema, Lemma, dan Corollary ===")  
    # Aksioma  
    aksioma = "Total sudut segitiga adalah 180 derajat"  
    print(f"Aksioma: {aksioma}")  
  
    # Teorema  
    teorema = "Segitiga siku-siku memiliki sudut 90 derajat"  
    print(f"Teorema: {teorema}")  
  
    # Lemma  
    lemma = "Sudut di seberang sisi terpanjang dalam segitiga siku-siku adalah 90 derajat"  
    print(f"Lemma: {lemma}")  
  
    # Corollary  
    corollary = "Jika segitiga memiliki sudut 90 derajat, maka segitiga tersebut adalah segitiga siku-siku"  
    print(f"Corollary: {corollary}")  
    print("=====\n")
```

PRAKTIKUM Bagian 10: Studi Kasus dengan Menerapkan Operator dalam Teori Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas and Corollaries

Langkah	Praktikum
4.	Bikondisional (Bi-implikasi) Di sebuah perusahaan, seorang karyawan dianggap layak mendapat bonus jika dan hanya jika ia bekerja lebih dari 8 jam sehari dan tidak pernah terlambat. Jika kondisi ini dipenuhi, maka karyawan tersebut mendapat bonus. Hal ini merupakan contoh dari <i>bikondisional</i> (bi-implikasi), di mana kedua kondisi saling terkait. Ketikkan code berikut dan tampilkan hasilnya.  <pre>def opbiconditional(work_hours, is_late): # Karyawan mendapatkan bonus jika bekerja lebih dari 8 jam dan tidak terlambat return (work_hours > 8) == (not is_late) # Studi kasus work_hours = 9 # Jam kerja lebih dari 8 is_late = False # Tidak terlambat # Penerapan result = opbiconditional(work_hours, is_late) if result: print("Karyawan layak mendapatkan bonus.") else: print("Karyawan tidak layak mendapatkan bonus.")</pre>
5.	Inferensi Jika hujan turun, maka tanah akan basah. Saat kita melihat bahwa tanah basah, maka kita dapat menyimpulkan bahwa hujan turun. Hal ini adalah contoh inferensi. Ketikkan code berikut dan tampilkan hasilnya.

	 <pre> def opinferensi(is_raining): # Jika hujan, maka tanah basah if is_raining: return "Tanah basah" else: return "Tanah kering" # Studi kasus is_raining = True # Hujan turun # Penerapan result = opinferensi(is_raining) print(f"Hasil inferensi: {result}") </pre>
6.	Argumen Ketika diberikan dua argumen pernyataan sebagai berikut. • Jika nilai ujian lebih dari 70, maka Anda lulus. • Anda memiliki nilai 85. • Dengan kedua argumen ini, kita dapat menyimpulkan bahwa Anda lulus. Ketikkan code berikut dan tampilkan hasilnya.  <pre> def opargumen(nilai): # Argumen: Jika nilai lebih dari 70, maka lulus if nilai > 70: return "Lulus" else: return "Tidak lulus" # Studi kasus nilai = 85 # Nilai lebih dari 70 # Penerapan result = opargumen(nilai) print(f"Hasil argumen: {result}") </pre>
7.	Aksioma, Teorema, Lemma, dan Corollary • Aksioma: Sebuah aturan dasar yang diterima tanpa bukti, misalnya "Setiap angka genap dapat dibagi 2."

- **Teorema:** Pernyataan yang dapat dibuktikan berdasarkan aksioma, misalnya "Jumlah dua bilangan genap adalah bilangan genap."
- **Lemma:** Pernyataan kecil yang digunakan untuk membuktikan teorema, misalnya "Bilangan genap adalah kelipatan dari 2."
- **Corollary:** Hasil langsung dari teorema, misalnya "Jika dua bilangan ganjil dijumlahkan, maka hasilnya bilangan genap."

Ketikkan code berikut dan tampilkan hasilnya.

```
Operator Axioms Theorems.py

# Aksioma: Setiap angka genap dapat dibagi 2
def opaksioma(n):
    return n % 2 == 0

# Lemma: Bilangan genap adalah kelipatan dari 2
def oplemma(n):
    return opaksioma(n)

# Teorema: Jumlah dua bilangan genap adalah bilangan genap
def opteorema(a, b):
    return opaksioma(a + b)

# Corollary: Jika dua bilangan ganjil dijumlahkan, maka hasilnya bilangan genap
def opcorollary(a, b):
    return opaksioma(a + b)

# Studi kasus
a = 4 # Bilangan genap
b = 6 # Bilangan genap

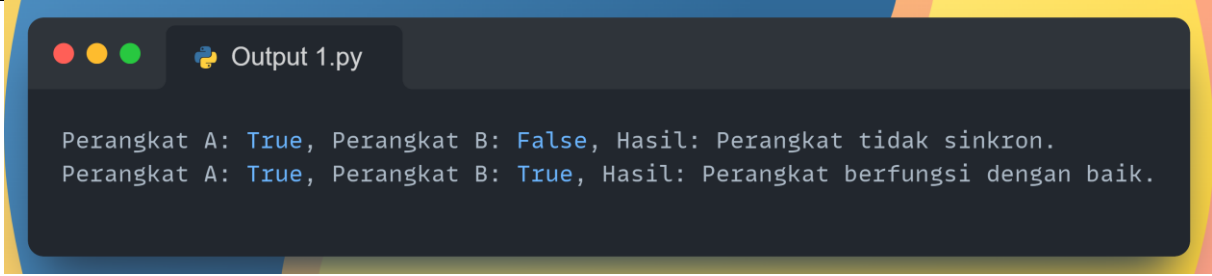
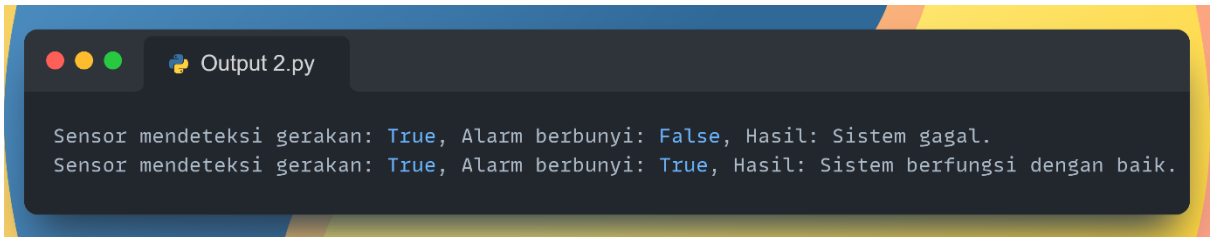
# Penerapan
print(f"Aksioma: {a} adalah bilangan genap? {opaksioma(a)}")
print(f"Lemma: {b} adalah kelipatan 2? {oplemma(b)}")
print(f"Teorema: Jumlah {a} dan {b} adalah bilangan genap? {opteorema(a, b)}")

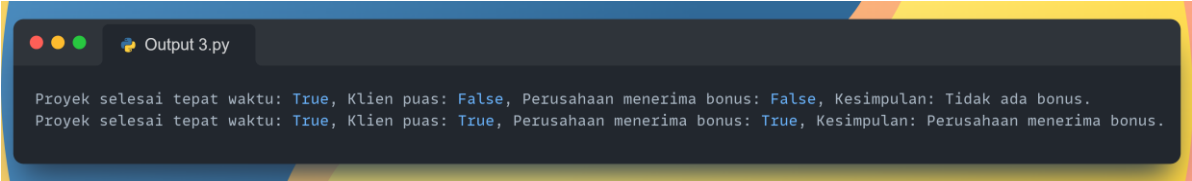
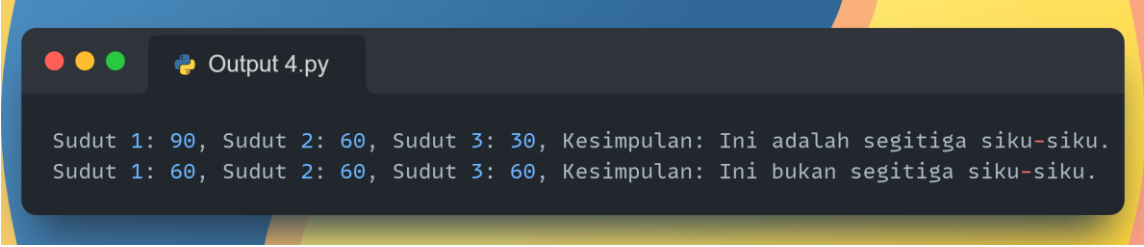
# Contoh bilangan ganjil
c = 3 # Bilangan ganjil
d = 5 # Bilangan ganjil
print(f"Corollary: Jumlah {c} dan {d} adalah bilangan genap? {opcorollary(c, d)}")
```



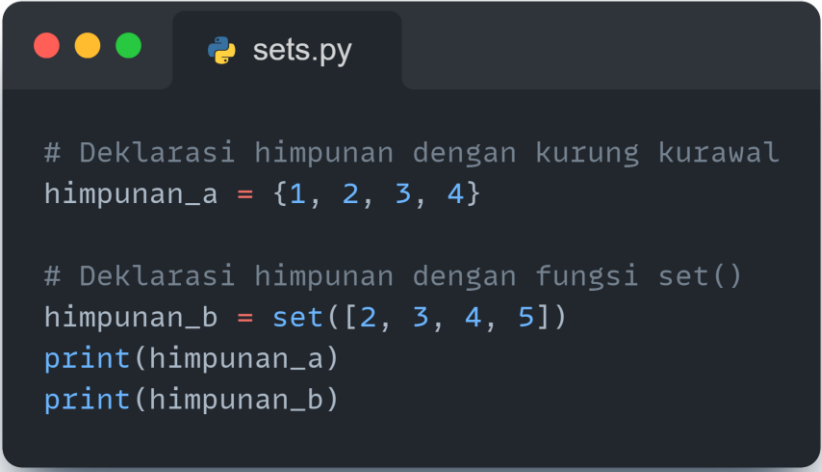
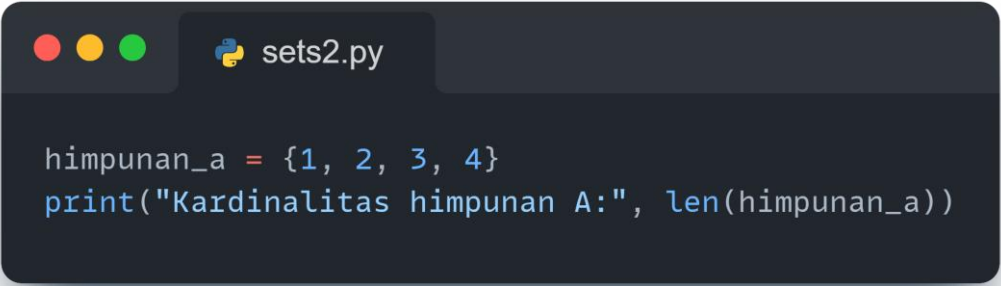
PRAKTIKUM Bagian 11: Tugas Praktikum Biconditional (Bi-implication), Inference, Arguments, Axioms, Theorems, Lemmas and Corollaries

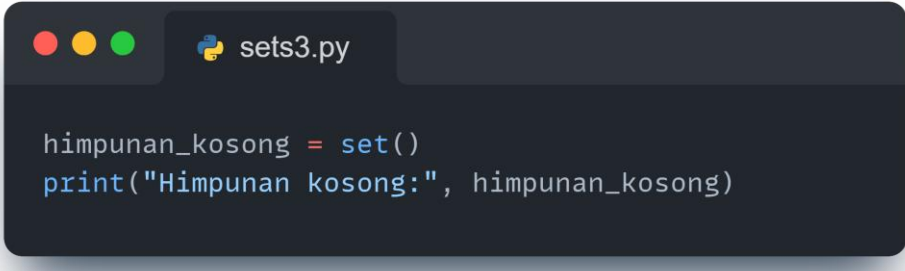
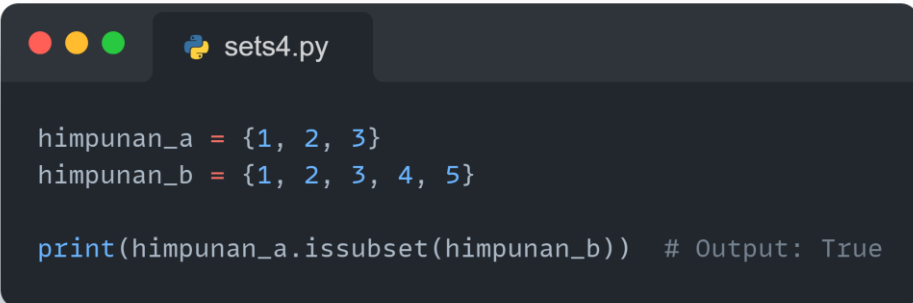
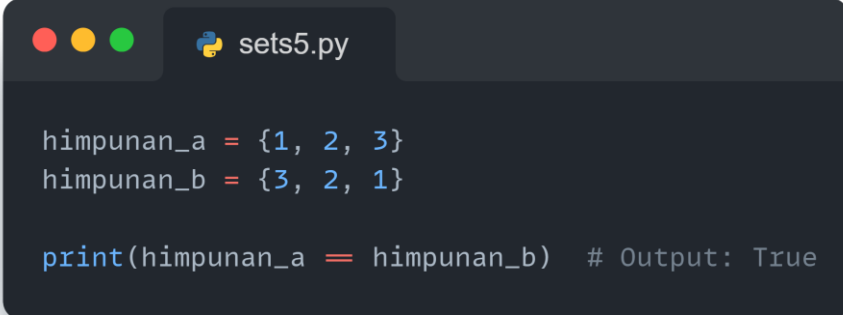
Langkah	Praktikum
1.	Pada Praktikum Bagian 2 (Langkah 1), ubah nilai input menjadi, • Perangkat A = True • Perangkat B = True dan • Perangkat A = False • Perangkat B = False Tampilkan dan analisis hasilnya.
2.	Pada Praktikum Bagian 2 (Langkah 2), ubah nilai input menjadi, • Sensor Gerak = True • Alarm Bunyi = True dan • Sensor Gerak = False • Alarm Bunyi = True Tampilkan dan analisis hasilnya.
3.	Pada Praktikum Bagian 2 (Langkah 3), ubah nilai input menjadi, • Proyek Selesai Tepat Waktu = True • Klien Puas = True • Perusahaan Terima Bonus = True Tampilkan dan analisis hasilnya.
4.	Tugas Penerapan Teori Bikondisional dalam Jaringan Komputer Pengelolaan jaringan yang terdiri dari dua perangkat (p dan q). Kedua perangkat ini hanya akan berfungsi dengan benar jika kedua perangkat menyala secara bersamaan atau keduanya mati. Implementasikan konsep Bikondisional untuk menguji apakah kedua perangkat bekerja secara sinkron. Buat fungsi Python yang menerima status perangkat p dan q (True atau False) dan mengecek apakah keduanya menyala/mati secara bersamaan. Tampilkan hasilnya dan analisis seperti format berikut. • Jika keduanya sama: "Perangkat berfungsi dengan baik." • Jika salah satu perangkat menyala dan yang lainnya mati: "Perangkat tidak sinkron."

	 <pre> Output 1.py Perangkat A: True, Perangkat B: False, Hasil: Perangkat tidak sinkron. Perangkat A: True, Perangkat B: True, Hasil: Perangkat berfungsi dengan baik. </pre>
5.	Tugas Penerapan Teori Inferensi pada Sistem Alarm Keamanan Pada sebuah sistem alarm keamanan, sensor gerak (p) mengaktifkan alarm (q). Jika sensor mendeteksi gerakan, maka alarm harus berbunyi. Implementasikan Inferensi untuk memastikan bahwa jika sensor mendeteksi gerakan, maka alarm harus berbunyi. Buat fungsi yang menerima input apakah sensor mendeteksi gerakan (True/False) dan apakah alarm berbunyi (True/False). Gunakan pernyataan inferensi logika untuk memeriksa apakah sistem bekerja sesuai aturan. Tampilkan hasilnya dan analisis seperti format berikut. • Jika inferensi gagal (sensor mendeteksi gerakan tapi alarm tidak berbunyi), • cetak "Sistem gagal." Jika inferensi valid, cetak "Sistem berfungsi dengan baik."  <pre> Output 2.py Sensor mendeteksi gerakan: True, Alarm berbunyi: False, Hasil: Sistem gagal. Sensor mendeteksi gerakan: True, Alarm berbunyi: True, Hasil: Sistem berfungsi dengan baik. </pre>
6.	Tugas Penerapan Teori Argumen dalam Proses Bisnis Dalam sebuah perusahaan, • Jika proyek selesai tepat waktu (p), klien akan puas (q). • Jika klien puas (q), perusahaan akan menerima bonus (r). Pergunakan dua premis ini untuk menyimpulkan bahwa jika proyek selesai tepat waktu, maka perusahaan akan menerima bonus. Implementasikan konsep Argumen dengan dua premis dan satu kesimpulan. • Buat fungsi yang menerima input status proyek (True/False), kepuasan klien (True/False), dan bonus perusahaan (True/False). • Periksa kesesuaian antara premis dan kesimpulan.

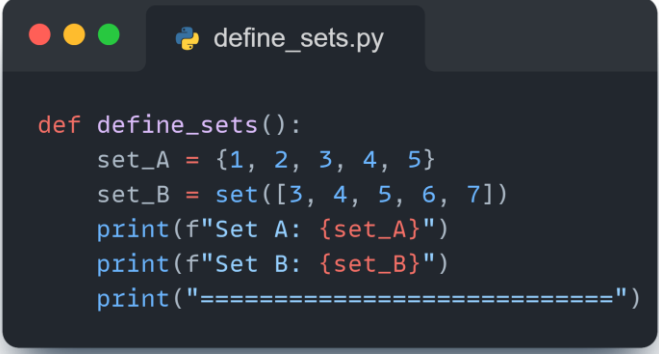
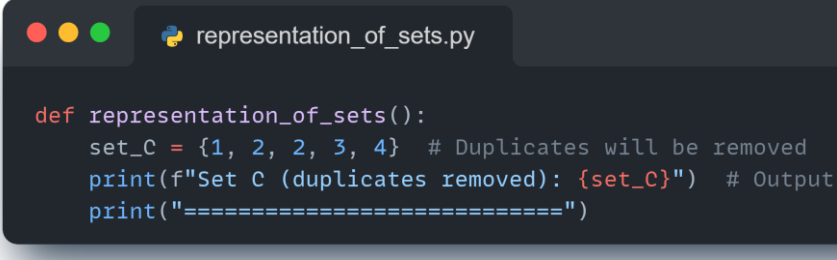
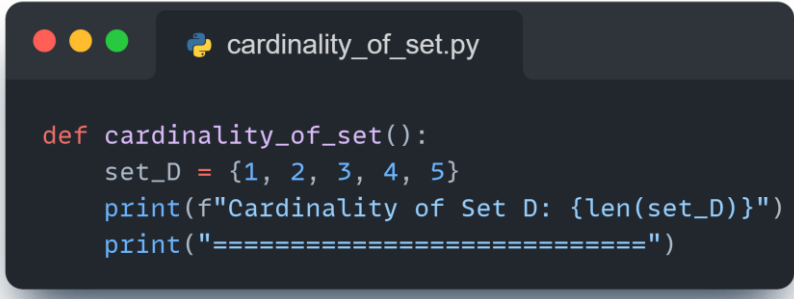
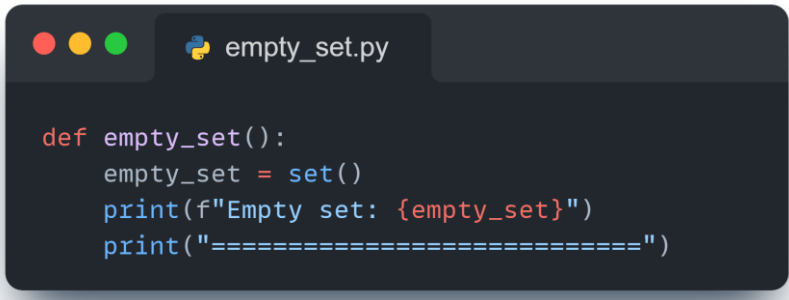
	Tampilkan hasilnya dan analisis dengan format. • Jika kesimpulan benar, cetak "Kesimpulan: Perusahaan menerima bonus." • Jika kesimpulan salah, cetak "Kesimpulan: Tidak ada bonus." 
7.	Tugas Penerapan Teori Aksioma, Teorema, Lemma, dan Corollary dalam Geometri Terdapat beberapa pernyataan dalam geometri, • Aksioma: "Total sudut dalam segitiga adalah 180 derajat." • Teorema: "Dalam segitiga siku-siku, salah satu sudutnya adalah 90 derajat." • Lemma: "Sudut di seberang sisi terpanjang dalam segitiga siku-siku adalah 90 derajat." • Corollary: "Jika segitiga memiliki sudut 90 derajat, maka itu adalah segitiga siku-siku." Implementasikan konsep Aksioma, Teorema, Lemma, dan Corollary dalam fungsi Python, Tampilkan hasilnya dan analisis dengan format. • Buat satu fungsi yang mengecek apakah sebuah segitiga adalah segitiga siku-siku berdasarkan nilai ketiga sudut. • Jika segitiga adalah segitiga siku-siku, cetak kesimpulan dari Teorema dan Corollary. Jika tidak, cetak "Ini bukan segitiga siku-siku." 

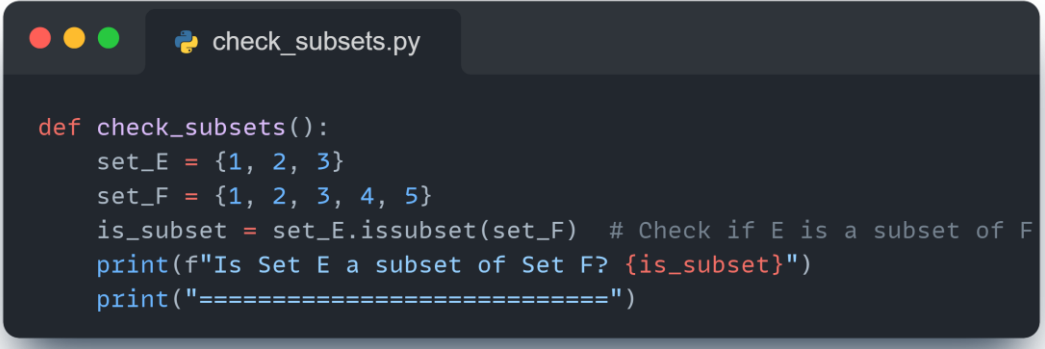
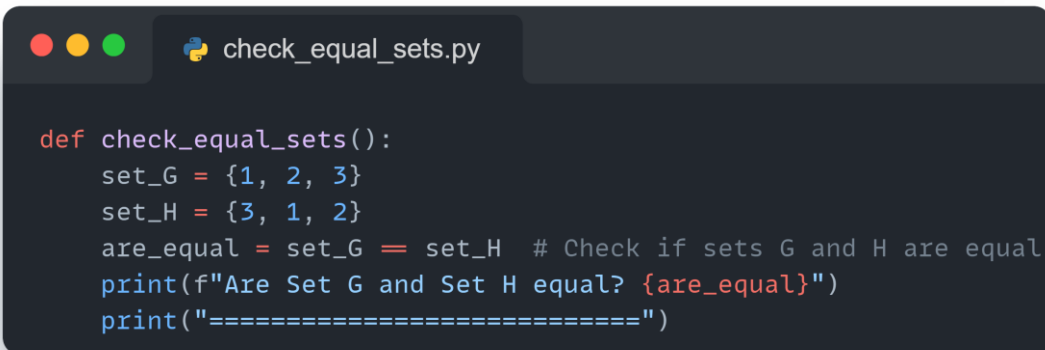
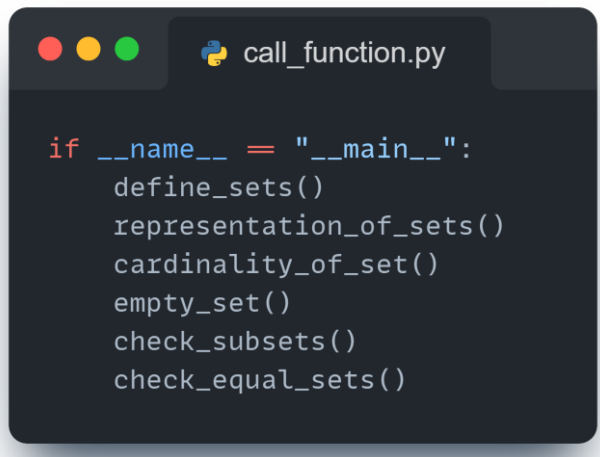
PRAKTIKUM Bagian 12: Penerapan Konsep Definition of Sets., Representation of Sets., Cardinality., Empty Set., Subsets., Equal Sets.

Langkah	Praktikum
1.	Pengantar Teori Himpunan Himpunan adalah kumpulan objek yang berbeda dan tidak berurutan. Dalam Python, himpunan diwakili dengan kurung kurawal {} atau fungsi set (). Elemen-elemen dalam himpunan tidak boleh ada yang duplikat, dan tipe data ini sangat berguna untuk operasi seperti pengujian keanggotaan, penggabungan, dan perpotongan. Representasi Himpunan dalam Python Himpunan dalam Python dapat dideklarasikan dengan dua cara: 1. Menggunakan kurung kurawal {}. 2. Menggunakan fungsi set (). Ketikkan code berikut dan analisis hasilnya.  <pre># Deklarasi himpunan dengan kurung kurawal himpunan_a = {1, 2, 3, 4} # Deklarasi himpunan dengan fungsi set() himpunan_b = set([2, 3, 4, 5]) print(himpunan_a) print(himpunan_b)</pre>
2.	Kardinalitas Himpunan Kardinalitas adalah jumlah elemen dalam sebuah himpunan. Dalam Python, dapat menghitung kardinalitas menggunakan fungsi len (). Ketikkan code berikut dan analisis hasilnya.  <pre>himpunan_a = {1, 2, 3, 4} print("Kardinalitas himpunan A:", len(himpunan_a))</pre>

3.	Himpunan Kosong (Empty Set) Himpunan kosong adalah himpunan yang tidak mengandung elemen. Dalam Python, himpunan kosong didefinisikan dengan fungsi set (). Ketikkan code berikut dan analisis hasilnya.  <pre>himpunan_kosong = set() print("Himpunan kosong:", himpunan_kosong)</pre>
4.	Sub himpunan (Subsets) Sebuah himpunan A dikatakan subhimpunan dari himpunan B jika semua elemen A juga terdapat di dalam B. Python menyediakan metode. issubset () untuk memeriksa apakah suatu himpunan adalah subhimpunan dari himpunan lain. Ketikkan code berikut dan analisis hasilnya.  <pre>himpunan_a = {1, 2, 3} himpunan_b = {1, 2, 3, 4, 5} print(himpunan_a.issubset(himpunan_b)) # Output: True</pre>
5.	Himpunan Sama (Equal Sets) Dua himpunan dikatakan sama jika mereka mengandung elemen yang sama. Dalam Python, kesamaan himpunan dapat diperiksa dengan operator ==. Ketikkan code berikut dan analisis hasilnya.  <pre>himpunan_a = {1, 2, 3} himpunan_b = {3, 2, 1} print(himpunan_a == himpunan_b) # Output: True</pre>

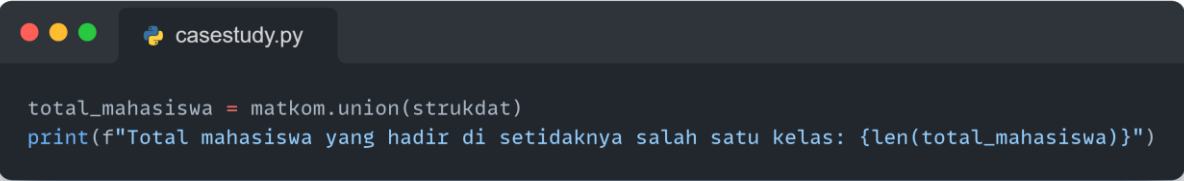
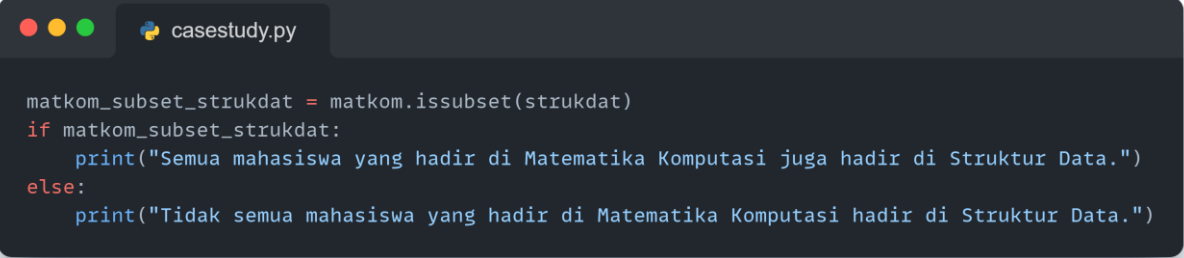
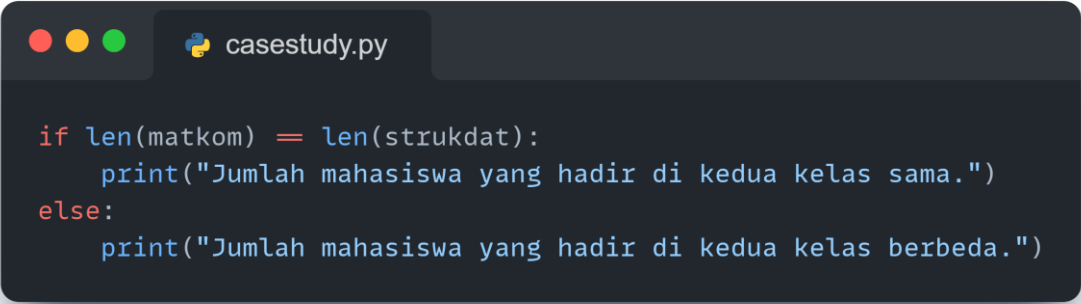
PRAKTIKUM Bagian 13: Penerapan dalam Python Definition of Sets., Representation of Sets., Cardinality., Empty Set., Subsets., Equal Sets.

Langkah	Praktikum
1.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def define_sets(): set_A = {1, 2, 3, 4, 5} set_B = set([3, 4, 5, 6, 7]) print(f"Set A: {set_A}") print(f"Set B: {set_B}") print("=====")</pre>
2.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def representation_of_sets(): set_C = {1, 2, 2, 3, 4} # Duplicates will be removed print(f"Set C (duplicates removed): {set_C}") # Output print("=====")</pre>
3.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def cardinality_of_set(): set_D = {1, 2, 3, 4, 5} print(f"Cardinality of Set D: {len(set_D)}") print("=====")</pre>
4.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def empty_set(): empty_set = set() print(f"Empty set: {empty_set}") print("=====")</pre>

5.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def check_subsets(): set_E = {1, 2, 3} set_F = {1, 2, 3, 4, 5} is_subset = set_E.issubset(set_F) # Check if E is a subset of F print(f"Is Set E a subset of Set F? {is_subset}") print("=====")</pre>
6.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>def check_equal_sets(): set_G = {1, 2, 3} set_H = {3, 1, 2} are_equal = set_G == set_H # Check if sets G and H are equal print(f"Are Set G and Set H equal? {are_equal}") print("=====")</pre>
7.	Ketikkan code berikut, running dan analisis hasilnya.  <pre>if __name__ == "__main__": define_sets() representation_of_sets() cardinality_of_set() empty_set() check_subsets() check_equal_sets()</pre>

PRAKTIKUM Bagian 14: Studi Kasus dalam Teori Definition of Sets., Representation of Sets., Cardinality., Empty Set., Subsets., Equal Sets.

Langkah	Praktikum
1.	Mahasiswa yang hadir di kedua kelas Penggunaan operasi intersection (matkom. intersection(strukdat)) untuk menemukan siapa saja mahasiswa yang hadir di kedua kelas, yaitu Matematika Komputasi dan Struktur Data. <pre> casestudy.py matkom = {"Deny", "Budi", "Citra", "Dewi", "Eko"} strukdat = {"Budi", "Citra", "Eko", "Farhan", "Gita"} </pre> <pre> casestudy.py mahasiswa_kedua_kelas = matkom.intersection(strukdat) print(f"Mahasiswa yang hadir di kedua kelas: {mahasiswa_kedua_kelas}") </pre>
2.	Mahasiswa yang hanya hadir di salah satu kelas Operasi symmetric_difference (matkom. symmetric_difference(strukdat)) digunakan untuk menemukan mahasiswa yang hanya hadir di salah satu dari kedua kelas, tetapi tidak di keduanya. <pre> casestudy.py mahasiswa_satu_kelas = matkom.symmetric_difference(strukdat) print(f"Mahasiswa yang hanya hadir di salah satu kelas: {mahasiswa_satu_kelas}") </pre>
3.	Total mahasiswa yang hadir di setidaknya salah satu kelas Operasi union (matkom. union(strukdat)) digunakan untuk menggabungkan kehadiran dari kedua kelas, sehingga bisa mengetahui jumlah total mahasiswa yang hadir di setidaknya satu kelas.

	 <pre>total_mahasiswa = matkom.union(strukdat) print(f"Total mahasiswa yang hadir di setidaknya salah satu kelas: {len(total_mahasiswa)}")</pre>
4.	Cek apakah semua mahasiswa Matematika Komputasi hadir di Struktur Data Fungsi subset check (<code>matkom.issubset(strukdat)</code>) memeriksa apakah semua mahasiswa dari kelas Matematika Komputasi juga hadir di kelas Struktur Data. Jika ya, maka hasilnya True, jika tidak maka False.  <pre>matkom_subset_strukdat = matkom.issubset(strukdat) if matkom_subset_strukdat: print("Semua mahasiswa yang hadir di Matematika Komputasi juga hadir di Struktur Data.") else: print("Tidak semua mahasiswa yang hadir di Matematika Komputasi hadir di Struktur Data.")</pre>
5.	Cek apakah kehadiran di kedua kelas sama Dengan membandingkan jumlah mahasiswa yang hadir di kedua kelas menggunakan <code>len(matkom) == len(strukdat)</code>, dapat memeriksa apakah jumlah mahasiswa yang hadir di kedua kelas sama atau tidak. Hasil perbandingan ini akan menunjukkan apakah kedua kelompok memiliki ukuran yang sama.  <pre>if len(matkom) == len(strukdat): print("Jumlah mahasiswa yang hadir di kedua kelas sama.") else: print("Jumlah mahasiswa yang hadir di kedua kelas berbeda.")</pre>



PRAKTIKUM Bagian 15: Tugas Praktikum Definition of Sets., Representation of Sets., Cardinality., Empty Set., Subsets., Equal Sets.

Langkah	Praktikum
1.	Buatlah program Python yang menerima input nama mahasiswa untuk dua kelas (Seperti Matematika Komputasi dan Struktur Data). Kemudian, gunakan operasi set untuk menjawab pertanyaan-pertanyaan berikut: • Siapa saja mahasiswa yang hadir di kedua kelas? • Siapa saja mahasiswa yang hanya hadir di salah satu kelas? • Berapa banyak total mahasiswa yang hadir di setidaknya salah satu kelas?
2.	Modifikasi program yang telah dibuat untuk menambahkan dua mata kuliah tambahan (Algoritma dan Basis Data) dengan inputan nama yang berbeda. Lakukan analisis serupa untuk kedua mata kuliah tersebut dan tampilkan hasilnya.
3.	Buatlah program Python yang mengembangkan analisis kehadiran mahasiswa dalam tiga kelas, yaitu "Matematika Komputasi", "Struktur Data", dan "Algoritma". • Tampilkan mahasiswa yang hadir di semua kelas menggunakan operasi intersection. • Tampilkan mahasiswa yang hanya hadir di salah satu kelas saja menggunakan operasi symmetric difference. • Tampilkan total mahasiswa yang hadir di setidaknya salah satu kelas menggunakan operasi union. (Poin 1-3 gunakan kelas Struktur Data dan Algoritma) • Cek apakah semua mahasiswa yang hadir di kelas "Matematika Komputasi" juga hadir di kelas "Struktur Data" menggunakan fungsi subset check. • Cek apakah kedua kelompok kehadiran mahasiswa dari semua kelas tersebut memiliki jumlah yang sama (Equal Sets). (Poin 4-5 gunakan kelas Matematika Komputasi dan Struktur Data)

--#BeraksiBerprestasiBersinergi--